A Faster Tableau for TCTL

Abstract. A zone-based tableau for TCTL satisfiability is exponential in the size of the formula, and in practice the exponent has two quite different sources. We separate them by measurement: on the family that costs the most, the symbolic part of a node contributes nothing at all — there is exactly one zone per distinct discrete state — and the quarter of a million nodes are a quarter of a million combinations of which obligations are still outstanding. No abstraction, no widening and no inclusion test can reduce that number, because there is nothing there to abstract.

We show that a large part of it is an artefact of representation. When a node's goals fall into groups that mention disjoint symbols, those groups have disjoint formula-clocks, and the node is realisable exactly when each group is; deciding them separately replaces a product of state spaces by a sum. The theorem needs no condition on the zone, which we had expected it to. Applied at every node with a cache, and at the root as a preprocessing step, it turns 2^k into $O(k)$ on conjunctions of independent obligations: at $k = 20$, 4 195 583 nodes and 31.4 seconds become 240 nodes and under ten milliseconds.

We report what the same measurements rule out. Conflict-driven search, the obvious import from SAT, has nothing to work on: on the hardest instance every node is accepted and none is closed. Zone inclusion cannot fire, because the several zones a discrete state carries are disjoint cells rather than nested ones. We also record six further attempts that failed, with the reason each failed, and a soundness bug the evaluation harness found in the process.

1 Introduction

Satisfiability for TCTL asks whether a formula has a timed model at all, rather than whether a given one satisfies it. A tableau answers the question by building an AND–OR graph whose nodes carry a set of goals and a zone, and by looking for a successful marking of that graph [10,1]. The graph is finite because zones are compared modulo a max-constant abstraction, and its size is bounded by $2^{O(n)} \cdot M^{O(n^2)}$ for a formula of size n over constants up to M.

That bound has two factors, and the folklore reading is that the second is the one to attack: refine the abstraction, widen more aggressively, subsume zones against one another. We began there and got nowhere, so we measured instead. Counting distinct *discrete* states against total nodes separates the two factors cleanly, and on the family that costs the most the ratio is 1.00. Every node is a distinct combination of outstanding obligations, carrying one zone. Nothing in the symbolic machinery can help, because the symbolic part is not doing the multiplying.

What is multiplying is a product of independent obligations. A conjunction of k bounded eventualities over disjoint atoms has 2^k combinations of which are still outstanding, and the search enumerates all of them, though the k questions never interact. The same shape is what a verification engineer writes when checking several properties of a system at once.

Contribution. We show the enumeration is avoidable, and by how much.

- A *decomposition theorem* (§4): if a node's goals, context and pursuits split into groups mentioning disjoint symbols, the node is realisable exactly when each group is. Two directions, one by projecting a model and one by taking a product of timed automata in which delays synchronise and edges interleave. The theorem needs no condition relating the groups' clocks in the zone, which we had expected it to and state explicitly because the expectation is natural and wrong.
- An algorithm (§5) that applies it at every node, with a cache keyed on the component, so a component recurring under many parents is decided once. Two side conditions carry the weight: clock ownership must be read off the formulas rather than the live pursuits, and the symbols that couple two groups include data variables, not only atomic propositions. Getting either wrong makes an unsatisfiable formula satisfiable, and we report both as measured failures.
- A measurement-driven account (§3, §7) of what does *not* work: conflict-driven search, zone subsumption, coarser LU abstraction, and four more. Each was implemented and measured rather than argued about, and each is reported with the structural reason it failed.

Results. On conjunctions of independent obligations the growth changes from 2^k to $O(k)$: at $k = 20$, 4 195 583 nodes and 31.4 seconds become 240 nodes and under ten milliseconds. On a suite of 5000 formulas across ten families the decomposing search returns exactly the same verdicts as the baseline, and it costs between two and six per cent on formulas that do not decompose. Independently of the decomposition, profiling removed a quartic step and two redundant closures from the zone layer, which is worth a factor of seven on its own.

A bug, and how it was found. The evaluation harness was built to carry expected verdicts wherever a window-containment argument settles them. It reported 39 wrong answers on a 700-case sweep. The cause was in the calculus rather than the implementation: the delay rule split a clock only at its own window endpoints, so the onsets whose demand is unsatisfiable sat inside a cell that also held satisfiable ones. We give the fix in §6. It is the reason we think a satisfiability tool of this kind needs an oracle-carrying benchmark and not only a regression suite.

2 Background

We fix a finite set of atomic propositions and interpret TCTL over the timed transition system induced by a timed automaton, in the standard way [2,6]. A

Table 1. Which factor is multiplying. The first row is a conjunction of 16 independent bounded eventualities, the others nested AG(EF $\cdots$) towers.

instance	shapes	nodes	zones per shape
16 conjoined $\mathrm{EF}_{[a_j,b_j]}\, p_j$	262 204	263 162	**1.00**
depth-7 $\mathrm{AG}(\mathrm{EF}_{[0,2]} \cdots)$ tower	5 841	10 785	1.85
4-level tower, varied windows	2 635	17 370	6.59

time bound I is a non-empty interval with integer endpoints; formulas are in negation normal form over the timed until and release modalities, and AG, EF and their relatives are the usual abbreviations. Satisfaction is evaluated over time-divergent runs Runs^{∞}.

A *zone* is a set of clock valuations defined by difference constraints and stored as a difference-bound matrix [3,9]. Zones are closed under intersection with a single-clock guard, reset, and time-successor, and are compared modulo max-constant extrapolation, which is what makes the set of reachable zones finite.

The tableau. A node is a tuple $N = \langle \Gamma, \mathcal{X}, Z, \mathcal{O}, \mathcal{D} \rangle$: the goals Γ to be satisfied at this configuration, the context $\mathcal{X}$ of formulas that must hold at every successor, a zone Z, a map $\mathcal{O}$ allocating a dedicated *formula-clock* x_μ to each active timed modality μ, and a set $\mathcal{D}$ of deadlines on those clocks. Rules saturate the propositional structure, launch a *pursuit* $\mu^\bullet$ when a timed modality is demanded, unfold pursuits step by step, and advance time. A node is *realisable* when some timed transition system satisfies $\bigwedge \Gamma \wedge \bigwedge \mathcal{X}$ at every valuation of its zone. The calculus is sound and complete, so a node lies in the greatest successful marking exactly when it is realisable; this is the only property of the calculus the present paper uses, and we use it in one place, to move between "realisable" and "accepted by the search".

Because a clock belongs to the modality that owns it, two formulas over disjoint atoms have disjoint formula-clocks. That observation is the whole of §4's hypothesis, and it is worth stating here because it is what makes the atom-level test in §5 sufficient to conclude something about clocks.

3 Where the cost is

The bound $2^{O(n)} \cdot M^{O(n^2)}$ has two factors. We separate them by counting, for a run, the number of distinct *discrete shapes* — nodes taken up to their zone and their edge markers — against the number of nodes.

Table 1 is the paper's starting point. On the most expensive instance the zone contributes nothing: one zone per discrete shape, and the 263 162 nodes are 263 162 distinct discrete states. Abstraction, widening and inclusion all act on the third column, and on the row that costs the most the third column is 1.00.

Table 2. How nodes end. Conflict-driven techniques need the middle columns.

instance	accepted	rejected	closed
16 conjoined eventualities (SAT)	262 975	**0**	**0**
4-level tower (SAT)	12 928	1 014	790
depth-6 tower (SAT)	4 281	609	385
$\mathrm{AG}(\mathrm{EF}_{[3,4]}p) \wedge \mathrm{AG}_{[4,5]}\neg p$ (UNSAT)	161	108	53

What the discrete states are. Instrumenting the pursuit vector on a depth-6 tower gives at most 12 simultaneous pursuits, exactly two per nesting level, with no modality duplicated, and a histogram of $|P|$ over the nodes of $5, 22, 77, 172, 323, 545, 824, 952, 903, 803, 485, 167, 2$ binomial, peaked in the middle. That is the signature of a search ranging over the *subsets* of a set of independent obligations. For $\bigwedge_{j<k} \mathrm{EF}_{[a_j,b_j]}\, p_j$ the reading is immediate — each conjunct is outstanding or witnessed, the conjuncts do not interact, and the node records the whole vector of k bits. Measured node counts follow: 1 247, 16 863, 66 179, 262 975, 1 049 619 for $k = 8, 12, 14, 16, 18$.

There is nothing to learn from. The natural import from SAT and SMT is conflict-driven search. Counting how nodes end says it has no purchase.

On the hardest instance every node is accepted and none is closed. The search is not thrashing through contradictory subtrees that a learned clause would cut; it is enumerating states that are all consistent. Asking a technique for *failing faster* to speed up a search that never fails is the wrong request. What is needed is a technique for not enumerating a product — and one exists in the same literature, in model counting rather than in solving [5,14].

4 Decomposition

Write $\mathrm{at}(\varphi)$ for the atomic propositions and data variables occurring in φ, and $\mathrm{cl}(\varphi)$ for the formula-clocks of the timed modalities occurring in it. For a set C of clocks, $Z{\downarrow}C$ is the zone obtained by existentially quantifying away the clocks outside C.

Definition 1 (Split). *A node $N = \langle \Gamma, \mathcal{X}, Z, \mathcal{O}, \mathcal{D}\rangle$ with pursuit set P* splits *into $N_1, \ldots, N_r$ ($r \geq 2$) when*

(S1) *Γ, $\mathcal{X}$ and P are partitioned among the N_i, and $\mathrm{at}(N_i) \cap \mathrm{at}(N_j) = \emptyset$ for $i \neq j$;*
(S2) *every clock constrained in Z belongs to exactly one $\mathrm{cl}(N_i)$;*

and N_i carries $Z{\downarrow}\mathrm{cl}(N_i)$, the deadlines on its own clocks, and the phase of N.

There is deliberately no condition on the zone, and this is worth dwelling on because the natural guess is that there should be. A reset leaves finite difference constraints in the matrix, so Z can relate x and y even when the modalities owning them share no atom; it is tempting to require Z to be the product of its projections and to refuse the split otherwise. That condition is sufficient and unnecessary — Remark 1 — and we implemented it before noticing.

Theorem 1 (Decomposition). *If N splits into $N_1, \ldots, N_r$, then N is realisable if and only if every N_i is.*

Proof. ($\Rightarrow$) Let $\mathcal{T}$ realise N at location s. Fix i and let $\mathcal{T}_i$ be $\mathcal{T}$ with its labelling restricted to $\mathrm{at}(N_i)$ and its clocks to $\mathrm{cl}(N_i)$. Every formula of N_i mentions only atoms of $\mathrm{at}(N_i)$ and is evaluated against clocks of $\mathrm{cl}(N_i)$, and the satisfaction relation inspects a configuration only through those two, so the truth of each is unchanged by the restriction; the runs and their delays are unchanged, so Runs^∞ is the same set and the path quantifiers range over it unchanged. Each $\nu \in Z{\downarrow}\mathrm{cl}(N_i)$ extends by definition of the projection to some $\nu' \in Z$, and $\mathcal{T}, (s, \nu')$ satisfies the formulas of N_i, hence so does $\mathcal{T}_i, (s, \nu)$.

($\Leftarrow$) Let $\mathcal{T}_i$ be induced by a timed automaton $\mathcal{A}_i$ over $\mathrm{cl}(N_i)$ realising N_i. The clock sets are pairwise disjoint, so form the product $\mathcal{A} = \prod_i \mathcal{A}_i$: locations are tuples, the invariant of a tuple is the conjunction of the components' invariants, the labelling is the union of theirs — consistent, the atom sets being disjoint — and edges interleave, each edge of one component lifting to an edge of the product that leaves the other components' locations and clocks untouched. Delays synchronise, and are admissible because each component's invariant permits them.

Runs project and lift. A run of $\mathcal{A}$ projects to a run of each $\mathcal{A}_i$ with the same delays, so it is non-Zeno exactly when its projections are. Conversely a run of one component and arbitrary runs of the others merge into a run of the product projecting to each: refine the delay sequences to their common refinement and interleave the discrete steps consistently with time. Non-blockingness of the components gives non-blockingness of the product, so Runs^∞ is non-empty everywhere.

Truth is componentwise. By induction on φ with $\mathrm{at}(\varphi) \subseteq \mathrm{at}(N_i)$. Literals are the definition of the product labelling and the boolean cases are immediate. For a formula under E, take the witnessing run of $\mathcal{A}_i$ and lift it against arbitrary runs of the others; for one under A, take any run of the product and project it, where the hypothesis applies. A timed modality adds only a constraint on the elapsed time of a position, which projection preserves because delays are shared.

The zone. Let $\nu \in Z$. By (S2) each coordinate belongs to exactly one component, so ν decomposes as $(\nu_1, \ldots, \nu_r)$ with $\nu_i \in Z{\downarrow}\mathrm{cl}(N_i)$, and (s_i, ν_i) realises N_i's formulas. By the previous paragraph the product configuration satisfies every formula of every N_i, that is, all of $\Gamma \cup \mathcal{X}$. As ν was arbitrary, $\mathcal{A}$ realises N. □

Remark 1 (No zone-product condition is needed). Let Z be constrained by $x - y < 1$ with x and y in different components. Both projections are universal and Z is not, so Z is a proper subset of the product of its projections. Splitting is still sound. Forwards, a valuation of a projection extends to one of Z and a component's formulas read only its own clocks. Backwards, each component is asked to be realisable over its *projection*, which is the larger demand: $Z \subseteq \bigcap_i Z{\downarrow}\mathrm{cl}(N_i)$, so a product model meeting the projections meets Z a fortiori.

Dropping the condition costs nothing and buys nothing measurable: across the whole suite it never once blocked a split that (S1) and (S2) allowed, and node

counts are identical with and without it. We report it because the condition is the natural first guess and a reader will otherwise wonder why it is absent.

Corollary 1. *If N splits, its verdict in the tableau is the conjunction of the verdicts of its components.*

Proof. The calculus is sound and complete, so a node lies in the greatest successful marking exactly when it is realisable; apply Theorem 1. □

5 The algorithm

Partitioning a node. Each goal, each context formula and each live pursuit is one *item*. Union–find over the items joins two whenever their symbol sets meet; that is (S1). Symbol sets are bitmasks, one bit per symbol of the input, memoised per formula. The test compares items at every node whose shape is new, and as sets of strings it measured about a third of the run on formulas that never split; as words it is one `&`. Union goes through a symbol-to-item index rather than over item pairs, which is $O(m \cdot \text{symbols per item})$ rather than $O(m^2)$.

Clock ownership, and a bug. Ownership is read off the *formulas*, not off the live pursuits: a modality still sitting in Γ already owns its formula-clock. Deriving ownership from pursuits alone leaves such a clock owned by nobody, and the construction then frees it out of every component's zone — a weakening. It is exactly what made $\mathrm{AG}(\mathrm{EF}_{[2,4]}p) \wedge \mathrm{AG}_{[4,7]}\neg p$ come back satisfiable: freeing the onset clock erases the constraint that distinguishes one onset from another. A clock owned by more than one group, or constrained and owned by none, aborts the split.

The symbols are not only propositions. A data variable couples its constraints exactly as an atom does. $\{x > 3\}$ and $\{x < 2\}$ mention no proposition, and grouping by propositions alone puts them in different components, each satisfiable alone. Twelve regression checks of the form $\{x \le 5\} \wedge \{x \ge 10\}$ failed on that, and no instance of the 5000-formula suite did, having no data variables in it: a gap in our own benchmark, found by the regression suite rather than by the benchmark.

Degenerate components. A component mentioning no symbol at all — a bare $\top$ or $\bot$ — is folded into the first real one rather than split off. Peeling those off is all cost and no benefit.

Deciding, and caching. Each component is keyed on its node key and looked up in a cache shared with every sub-search; on a miss a fresh search runs from that node and stores the verdict, and one false component rejects. The cache is what makes this component *caching* rather than component splitting: the same component recurs under many parents, which is precisely why the undecomposed search enumerated their product. Recursion terminates because every component is strictly smaller than the node it came from.

Hooking into the search. The test runs before a node is first visited. A node that splits is marked settled with the conjunction of its components' verdicts and is never expanded: it becomes a leaf, exactly like a node whose own strongly connected component has already closed, which the search already knows how to consume. No cycle can run through it, which is sound because Corollary 1 decides it outright.

Not re-asking. Whether a node splits depends only on which formulas it carries, never on its zone, so a shape that did not split never will and its signature is remembered. Shapes repeat heavily — a nested tower has roughly half as many shapes as nodes — and without this the partition is recomputed at each of them. The signature is combined commutatively so that no sort is needed, since it is computed before the memo can be consulted and so is paid at every node; a collision can only make two shapes look alike and skip an available split, which costs an optimisation and never a verdict.

At the root. The same theorem applied once to the top-level conjunction is a preprocessing step, and it pays nothing per node. It sees nothing whose independence only emerges during the search. The two forms are complementary and compose: §7 reports that running both takes the better of the two on every shape measured.

6 Two fixes the benchmark forced

Neither belongs to the decomposition. Both were found by an evaluation harness that carries expected verdicts, and we report them because the first is a soundness bug in the calculus as published and the second is worth a factor of seven on its own.

Onsets between split points. Under a persistent context a modality is demanded at a continuum of onsets, and the tableau represents them by cells of the zone. The delay rule split a clock at the endpoints of its own window only. Under a ban on ψ over $[c, d]$, an onset at t demands ψ somewhere in $[t+a, t+b]$, and that demand is unsatisfiable exactly for $t \in [c-a,\ d-b]$; splitting the context clock at c and d alone leaves that interval inside a cell that also holds satisfiable onsets, and the cell's launch succeeds on one of those. On $\mathrm{AG}(\mathrm{EF}_{[3,4]}p) \wedge \mathrm{AG}_{[4,5]}\neg p$ the bad onset is the single point $t = 1$, no cell isolates it, and the formula — which is unsatisfiable — was reported satisfiable.

A clock must therefore also split at its own endpoints shifted by those of every *other* modality. The relation is co-liveness, not nesting: in the formula above the ban and the demand are conjuncts, and a version restricted to syntactic nesting left every one of the 39 wrong verdicts in a 700-case sweep in place. There are two shifts per pair of modalities and none depends on the magnitude of a constant, so the count of split points stays quadratic in the formula and flat in M: node counts are 281, 281, 281, 278 for $M = 4, 64, 1024, 65536$.

A quartic step and two redundant closures. Profiling put 93% of the run in the construction of a node's key. Eliminating the clocks a node no longer reads freed them one at a time, and each free re-closed the matrix, so the step was $O(K^4)$. Batching the frees makes it cubic — freeing only relaxes, and a closed matrix already carries the tightened value of $d[i][0]$, so the column each free writes can be read off the original matrix, with the one correction that a freed clock has lost its own upper bound.

The closure then turns out to be unnecessary altogether. Checking $d[i][j] \le d[i][l]+d[l][j]$ on the updated matrix given it held on the old one: a path through a freed clock is $d[i][0] + \infty = \infty$; a freed row is ∞ throughout; a freed column becomes $d[i][0]$, and old closure already gave $d[i][0] \le d[i][l] + d[l][0]$; and the reference row needs $d[0][l] + d[l][0] \ge 0$, which holds because a closed non-empty zone has no negative cycle. Max-constant extrapolation gets the same treatment: a zone none of whose bounds exceeds its clock's maximum is unchanged and arrived closed.

A third defect in the same layer was a correctness one. The minimal-graph packer writes whole records but not whole words, so with 16-bit bounds and an odd constraint count the final half-word was never written, and it sits inside the length the packer reports. The buffer being reused, the same zone encoded differently depending on what had been packed before it, and the search rebuilt the same node under several keys. Clearing the buffer makes the encoding a function of the zone, and removes 15% of the nodes on a nested tower. The symptom that exposed it is worth recording: node counts moved when an unrelated cache size changed, which is not something a correct encoding can do.

Together these take a 17-clock instance from 12.45 s to 1.41 s with identical verdicts.

7 Evaluation

Two searches are selectable at the command line: `v1` is the baseline with the optional optimisations off, `v2` adds both decomposition forms and goal subsumption. Both carry the soundness fix of §6, which is not an optimisation. All measurements are single-threaded on one machine; the medians of the suite families sit near 8–10 ms and are dominated by process start-up, so only $p95$, maxima and the large instances carry signal.

The family the theorem targets. Table 3 is $\bigwedge_{j<k} \mathrm{EF}_{[2+j,4+j]}\, p_j$.

The two forms compose. They see different things. Table 4 adds `buried`, which is $\mathrm{EF}_{[0,1]}(\bigwedge_j \mathrm{EF}_{[a_j,b_j]} p_j)$: the independence is real but sits under a modality, so the root is not a conjunction and the preprocessing form returns the undecomposed count exactly. Running both takes the better of the two everywhere.

Verdicts. A suite of 5000 formulas over ten structural families carries expected verdicts wherever a window-containment argument settles them. `v1` and `v2` agree

Table 3. Independent obligations. The ratio is not a constant: the change is $O(2^k) \to O(k)$, so any single figure quoted is a statement about which k was measured.

k	`v1` nodes	`v1` time	`v2` nodes	`v2` time	ratio
12	16 863	0.08 s	551	0.01 s	13×
16	262 975	1.35 s	192	0.00 s	349×
18	1 049 619	9.35 s	1 249	0.01 s	1058×
20	4 195 583	31.4 s	240	0.00 s	**8309**×

Table 4. Nodes under each form. `buried` is invisible to the root split; `wide` is where the per-node form pays bookkeeping the root split does not.

	none	root only	per-node	both
`wide` k=16	262 975	192	941	**192**
`wide` k=20	4 195 583	240	1 869	**240**
`buried` k=14	66 181	66 181	731	**731**
`mixed` k=14	132 774	176	750	**176**

on all 5000, and neither contradicts an expectation. A 700-case sweep over $\mathrm{AG}(\mathrm{EF}_{[a,b]}p) \wedge \mathrm{AG}_{[c,d]}\neg p$, whose verdict is decided by an inequality on (a, b, c, d), is exact for both. The regression suite passes 861/861 checks under each. A witness automaton is extracted and structurally validated for all 4141 satisfiable instances.

The cost where it does not fire. A second suite, of ten families of 220 instances, exists to break `v2` rather than to flatter it: families one symbol away from decomposing, chains whose transitive closure is a single component, independence buried under a nest, components that decompose again, deep towers, constants across six orders of magnitude, punctual intervals, data variables, and conjunction widths up to 80. Across all 2200 instances there is no wrong verdict, no crash, no `v2` timeout and no instance where `v2` is materially slower. `v1` exhausts its budget on 200 of them and `v2` answers every one, with median speed-ups of 63× on the widest conjunctions and 11× on recursively decomposing ones.

What it did find is a tax. On the families built so that decomposition never fires, `v2` was 9–11% slower, consistently enough at 0.1–0.3 s runtimes not to be measurement noise. Hoisting three per-call allocations out of the partition, indexing the symbol memo by dense formula identifier, and the two changes noted in §5 bring it to 2–6%: the ratio on a depth-9 tower moves from 0.91 to 0.98, and on the shared-symbol family from 0.89 to 0.94.

Threats to validity. The family that decomposes is one we wrote, and it has exactly the shape the technique targets; the defensible claim is about the mechanism and not about a speed-up on a workload. Conjunctions of independent properties are common in practice — a safety property and a liveness property over different signals — but this suite is not evidence for how common. The

root-level form is preprocessing, and splitting a top-level conjunction is close to running the tool once per conjunct; we report it separately for that reason. Finally, witness extraction is disabled under decomposition, a witness for a decomposed node being the product of its components' automata, which the extractor does not build.

8 Related work

Component caching. The technique is imported from exact model counting, where a DPLL search that splits the residual formula into variable-disjoint components and caches their counts is what makes #SAT tractable on structured inputs [5,14]. The transfer is not mechanical. There the components partition a propositional formula and the results are multiplied; here they partition a tableau node carrying a symbolic zone, the results are conjoined, and soundness needs a model-theoretic argument (Theorem 1) because the components must be recombined into a single timed model that agrees with all of them about the passage of time. The clock-ownership condition has no counterpart in the propositional setting.

What did not transfer. Clause learning, backjumping and the rest of the conflict-driven apparatus [8] require conflicts, and §3 measures none on the instances that cost the most. We record this because the reflex when a search is slow is to reach for CDCL, and here that reflex is measurably wrong.

Zone abstractions. Coarser abstractions than max-constant extrapolation are standard, in particular the LU bounds [7] and their refinements [12]. They do not apply here as the construction stands: the split points a clock is tested against are used as equalities by the delay rule, so a clock's lower and upper bound sets coincide and Extra_{LU} collapses to Extra_{M}. Reworking the splitting to be one-sided is the prerequisite, and we have not done it. Inclusion-based subsumption of zones, the passed-list test of a reachability tool, cannot fire either: the several zones a discrete state carries are the disjoint cells the delay rule produces, never nested ones.

TCTL satisfiability. The decision problem and its complexity are classical [2,4], with tree-automata constructions giving matching upper bounds [13]; discrete-time quantitative CTL has its own tableau tradition [11]. The present paper is not about the decision problem but about the cost of one construction for it, and everything here applies to any tableau whose nodes carry an independent set of obligations.

9 Conclusion

The cost of a zone-based TCTL tableau has two sources, and they need different treatment. We measured which one dominates, found that on the most expensive

instances it is the discrete one and that the symbolic machinery contributes a factor of exactly 1.00, and removed a large part of the discrete factor with a decomposition theorem imported in spirit from model counting. Conjunctions of independent obligations go from 2^k to $O(k)$.

What we could not move is nesting. An AG inside an EF inside an AG shares atoms all the way down, decomposition does not apply, and eight further attempts — zone subsumption, LU abstraction, forced stops, collapsing the pursuit record, goal subsumption, encoding AG as context, and treating single-polarity atoms as non-linking — either failed outright or bought a few per cent. Several failed instructively and are documented with the formula that breaks them. The last of them is worth stating here because the argument for it is so tempting: an atom occurring under a single polarity looks harmless, because a product of models could take the union of the labellings for it, and declaring such atoms non-linking would make conjunctions of obligations gated on a shared enabling signal decompose. It is unsound, and not because the monotonicity argument has a gap. Masking an atom out of the linking relation leaves it *occurring* in both components, and (S1) asks for disjoint atom sets, so the relaxation violates the theorem's hypothesis rather than weakening a side condition. A future attempt must rewrite the components so the shared atom really occurs in one of them only. Our present belief is that the remaining growth on nested shapes is the genuine $2^{O(n)}$ of the bound.

Two methodological points seem worth more than the speed-ups. First, the profile was wrong three times running: the bottleneck was never where the literature's emphasis on abstraction suggested, and each time it moved after being fixed. Second, an evaluation suite that carries expected verdicts found a soundness bug in the calculus that a regression suite of the usual kind had not, and a regression check found a soundness bug in the decomposition that the evaluation suite could not, because the suite had no data variables in it. The two are complementary and we would not now build one without the other.

References

1. Abuin, A., Bolotov, A., Hermo, M., Lucio, P.: One-Pass Context-Based Tableaux Systems for CTL and ECTL. In: Muñoz Velasco, E., Ozaki, A., Theobald, M. (eds.) 27th International Symposium on Temporal Representation and Reasoning (TIME 2020). Leibniz International Proceedings in Informatics (LIPIcs), vol. 178, pp. 14:1–14:20. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2020). `https://doi.org/10.4230/LIPIcs.TIME.2020.14`
2. Alur, R., Courcoubetis, C., Dill, D.: Model-checking for real-time systems. In: Proceedings of the 5th Annual IEEE Symposium on Logic in Computer Science (LICS). pp. 414–425 (1990)
3. Alur, R., Dill, D.L.: A theory of timed automata. Theoretical computer science **126**(2), 183–235 (1994)
4. Alur, R., Henzinger, T.A.: A really temporal logic. Journal of the ACM **41**(1), 181–203 (1994)
5. Bacchus, F., Dalmao, S., Pitassi, T.: Algorithms and complexity results for #sat and Bayesian inference. In: FOCS. pp. 340–351 (2003)

6. Baier, C., Katoen, J.P.: Principles of Model Checking. MIT Press (2008)
7. Behrmann, G., Bouyer, P., Larsen, K.G., Pelánek, R.: Lower and upper bounds in zone-based abstractions of timed automata. In: TACAS. LNCS, vol. 2988, pp. 312–326 (2004)
8. Biere, A., Heule, M., van Maaren, H., Walsh, T.: Handbook of satisfiability. Frontiers in Artificial Intelligence and Applications **185** (2009)
9. Bouyer, P.: An introduction to timed automata. In: Actes de l'École d'été ETR'05 (École Temps-Réel 2005). pp. 79–94. Nancy, France (2005), tutorial
10. Clarke, E.M., Grumberg, O., Peled, D.A.: Model Checking. MIT Press (1999)
11. Emerson, E.A., Mok, A.K., Sistla, A.P., Srinivasan, J.: Quantitative temporal reasoning. Real-Time Systems **4**, 331–352 (1992)
12. Herbreteau, F., Srivathsan, B., Walukiewicz, I.: Better abstractions for timed automata. In: LICS. pp. 375–384 (2012)
13. La Torre, S., Napoli, M.: Timed tree automata with an application to temporal logic. Acta Informatica **38**, 89–116 (2001)
14. Sang, T., Beame, P., Kautz, H.A.: Heuristics for fast exact model counting. In: SAT. LNCS, vol. 3569, pp. 226–240 (2005)